

# claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_793a5d86-1c4\agent-a4c34aa385ce4dcca.jsonl`  
- SHA-256: `c86f3877e3b02963dab1140ee58109e1b0edc90279fb86a10913eb8c93d02b80`  
- Source modified: `2026-07-02T08:18:34+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_793a5d86-1c4`  
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`
```

## Transcript

### 1. user (2026-07-02T08:17:51.478Z)

You are an ADVERSARIAL verifier. Independently re-check the fate verdict for OPEN PR #418 ("fix(api): /api/source downloads remote source ONCE into reusable cache (R1-03)", branch origin/fix/api-source-remote-cache) on Khelsutra/badminton-highlight-indexer. Try to REFUTE the analyst.

Analyst's verdict: recommended\_fate=CLOSE\_SUPERSEDED; superseded\_by=425 (confidence high).  
Analyst's master\_evidence: Master @ d00b4db already implements the entire R1-03 intent, landed by merged #425 (commit 6c767b5 "cache remote source material", which touched the exact same two files: backend/main.py + tests/test\_api\_annotations.py). Concrete master citations in backend/main.py: (a) source-cache dir helper ? lines 608-609 `\_source\_cache\_dir` returns `/source\_cache`; (b) per-video serialize lock + guard ? lines 600-601 `\_source\_cache\_locks`/`\_source\_cache\_locks\_guard`, lock accessor 638-640; (c) fetch-once-and-reuse with double-checked cache-hit, free-space guard, atomic publish, and partial cleanup ? `\_cached\_remote\_source` lines 652-677: cache-hit early return (655-656 and again under lock 659-660), `require\_free\_bytes(cache\_dir, need, stage="source-cache")` before any bytes move (663-664), fetch to a unique `.<name>.<uuid>.tmp` then `os.replace(tmp, cache\_path)` atomic publish (665-670), `finally` remove-partial (671-676); (d) get\_source rewrite ? local ref is identity passthrough `local = input\_ref.key` (731-732), remote goes through `\_cached\_remote\_source` (733-741) with HTTPException re-raised (507 guard preserved) and other failures surfaced as 502 (738-741); (e) proxy warming now unconditional for both local and remote (746-748), closing the 'remote-source proxying is a follow-up' note. Supporting helpers all present on master: `require\_free\_bytes` (backend/utils/freespace.py:49), `resolve\_under\_root` (backend/utils/paths.py:45), `\_remote\_input\_size\_bytes` (backend/main.py:1479). Master test test\_source\_caches\_remote\_fetch\_and\_warms\_proxy\_from\_cache (tests/test\_api\_annotations.py:67-108) asserts len(fetches)==1 across two Range requests, cache at source\_cache/vidremote.mp4, and proxy warmed from the cached copy.  
Analyst's conflict\_nature: CONFLICTING due to full same-region supersession, not adjacent edits. Both PR #418 and merged #425 rewrite the identical span of backend/main.py (helpers around lines 600-698 and the get\_source body ~700-748) and add tests to the same block of tests/test\_api\_annotations.py. #418 introduces `\_acquire\_source\_for\_serving` + `\_source\_fetch\_locks` + a per-video subdir cache; master already introduced the semantically equivalent `\_cached\_remote\_source` + `\_source\_cache\_locks` + flat video\_id-keyed cache in the same lines. The PR diff's base (old get\_source used `storage.acquire\_local\_input`/`input\_is\_local`) no longer exists on master, so it cannot apply. The conflict is entirely because master already did the same thing.  
Analyst's rationale: Merged #425 already implements the complete R1-03 fetch-once-into-source\_cache-and-reuse behavior in the exact files #418 targets, including atomic publish, per-video lock, free-space 507 guard, 502-on-failure, local passthrough, and remote proxy warming. #418's implementation is equivalent (a few details are actually handled slightly better on master via resolve\_under\_root and by not trusting the remote object name). The CONFLICTING status is pure same-region supersession, so a rebase would leave essentially nothing to add. Close as superseded by #425.

Already MERGED onto master (verify against actual master code, do not assume):

- #422 = fix for --output-dir clobbering config.json output.output\_dir (CODE\_MAP updated for it)
- #423 = docs: health-remediation plan added + DOC\_STATUS/README/NEXT\_STEPS registered; audit marked snapshot
- #424 = P2: /api/process idempotent by content hash (#340 re-bill fix)
- #425 = P3: /api/source resolves remote source into output-scoped source\_cache, reuses across Range requests, warms annotation proxy
- #426 = P4: bounded ffmpeg/ffmpeg subprocess timeouts on serving paths
- #427 = P5: clean remote-input scratch dirs after process/compile/assets/CLI; durable reports
- #428 = P1/P6: backend/storage/gcs.py lazy-import mpy typing fix

master HEAD = d00b4db "fix gcs storage lazy import typing (#428)"

Working dir: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer on master.  
 SHARED checkout ? READ-ONLY, no branch checkout, no working-tree changes.

Your job ? verify by looking at code yourself, do NOT take the analyst's word:

- If the verdict is CLOSE\_SUPERSEDED: the risk is closing a PR whose work is NOT actually on master (losing real work). Open the specific master file:line the analyst cited AND the PR diff (gh pr diff 418 -R Khelsutra/badminton-highlight-indexer). Confirm master truly implements the SAME intent (not just a similarly-named symbol). If the PR does anything master lacks (extra tests, a stricter guard, a different file), flag it ? that means NOT fully superseded.
- If the verdict is TAKE\_OVER\_\*: the risk is that the work IS already on master (would be redundant/conflicting) OR the change is buggy. Confirm master still lacks it, and sanity-check the change is correct.
- Default to skepticism: only agree=true if the evidence in code clearly supports it.

Return ONLY the structured object.

## 2. assistant (2026-07-02T08:17:55.010Z)

I'll independently verify the analyst's CLOSE\_SUPERSEDED verdict by examining both the PR diff and the actual master code.

## 3. user (2026-07-02T08:17:58.263Z)

```
d00b4db fix gcs storage lazy import typing (#428)
---PR DIFF---
diff --git a/backend/main.py b/backend/main.py
index 99246fe..078fa2b 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -620,13 +620,79 @@ def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
     _proxy_inflight.discard(video_id)

+# Per-video locks so concurrent FIRST requests for a cold remote source serialize on ONE
download
+# (the losers re-check and reuse the winner's copy) instead of each pulling the full object.
+_source_fetch_locks: "dict[str, threading.Lock]" = {}
+_source_fetch_locks_guard = threading.Lock()
+
+def _source_cache_dir(cfg) -> str:
+    """Where /api/source caches fetched REMOTE sources ? under the guarded local output
root."""
+    return os.path.join(output_base(cfg), "source_cache")
+
+def _acquire_source_for_serving(video_id: str, raw_ref: str, cfg) -> str:
+    """Resolve a video's recorded filepath to the LOCAL file ``/api/source`` streams.
+
+A local path is an identity passthrough (no copy, no temp ? byte-identical to serving the
recorded path directly). A REMOTE ref (``gs://``/``s3://``/``gdrive://``?) is fetched ONCE
into
+
``<output>/source_cache/<video_id>/<name>`` and REUSED: ``FileResponse`` serves Range, so
```

```

one
+   scrubbing ``<video>`` tag issues many independent requests to this URL ? before this cache
each
+   request re-downloaded the full object into a fresh, never-deleted temp dir (a cloud-egress
bill
+   plus a disk-fill of the appliance; audit R1-03).
+
+   Concurrency + crash safety: the download goes to a unique ``*.partial`` name inside the
cache
+   dir and is published with an atomic ``os.replace`` ? a truncated file can never sit at the
+   served name, and a failed fetch leaves nothing behind. Racing first requests serialize on a
+   per-video lock. The download is free-space guarded (D11) when the backend can ``stat`` the
+   object's size; the cached copy is a regenerable intermediate (safe to delete any time)."""
+   settings = _storage_settings(cfg)
+   ref = storage.resolve_input_ref(raw_ref, settings)
+   if ref.backend == "local":
+       return ref.key # identity ? today's local behaviour, byte-for-byte
+
+   vid_dir = os.path.join(_source_cache_dir(cfg), safe_output_filename(video_id))
+   try:
+       fname = safe_output_filename(ref.name or "in.mp4")
+   except ValueError: # a hostile/odd remote object name never dictates the local filename
+       fname = "in.mp4"
+   final = os.path.join(vid_dir, fname)
+   if os.path.isfile(final):
+       return final # cache hit ? no re-download, whatever the browser's Range pattern
+
+   with _source_fetch_locks_guard:
+       lock = _source_fetch_locks.setdefault(video_id, threading.Lock())
+   with lock:
+       if os.path.isfile(final):
+           return final # a concurrent first request fetched it while we waited on the lock
+       os.makedirs(vid_dir, exist_ok=True)
+       need = None
+       try: # sizing is best-effort ? a backend that can't stat still fetches (and fails
loudly)
+           need = int(storage.get_storage(ref.backend, config=settings).stat(ref).size)
+       except Exception: # noqa: BLE001 - stat is optional; require_free_bytes(None) is a no-
op
+           need = None
+       require_free_bytes(vid_dir, need, stage="source-cache download")
+       tmp = os.path.join(vid_dir, f".{fname}.{uuid.uuid4().hex}.partial")
+       try:
+           storage.fetch(ref.uri, tmp, config=settings)
+           os.replace(tmp, final) # atomic publish ? readers only ever see a COMPLETE file
+       finally:
+           if os.path.exists(tmp): # fetch (or publish) failed ? never leave a partial behind
+               try:
+                   os.remove(tmp)
+               except OSError:
+                   pass
+       return final
+
+
+   @app.get("/api/source/{video_id}")
+   def get_source(video_id: str, request: Request):
+       """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub. Serves
a
+       frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels still
keyed
+       to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy in the
-       background for next time ? non-blocking, so the first load is never delayed. Proxy warming
is
-       LOCAL-source only (remote-source proxying is a follow-up)."""
+       background for next time ? non-blocking, so the first load is never delayed. A REMOTE

```

```

source is
+   downloaded ONCE into ``<output>/source_cache/<video_id>/`` and every later request (each
browser
+   Range request hits this URL anew) streams that local copy ? never a re-download (R1-03)."""
    rec = request.app.state.db.get_video(video_id)
    if not rec or not rec.get("filepath"):
        raise HTTPException(status_code=404, detail="Video not found.")
@@ -641,20 +707,21 @@ def get_source(video_id: str, request: Request):
    if proxy_path and os.path.isfile(proxy_path):
        return FileResponse(proxy_path, media_type="video/mp4")

-   # 2) Otherwise resolve + serve the source.
+   # 2) Otherwise resolve + serve the source (remote ? the reused source-cache copy).
    try:
-       local = storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage", None))
-       is_local = storage.input_is_local(rec["filepath"], getattr(cfg, "storage", None))
-       except ValueError as e: # unsupported scheme etc. ? a clean client error, never a 500
+       local = _acquire_source_for_serving(video_id, rec["filepath"], cfg)
+   except HTTPException:
+       raise # already a clean client answer (e.g. the 507 free-space guard)
+   except ValueError as e: # unsupported scheme / unsafe id ? a clean client error, never a
500
        raise HTTPException(status_code=400, detail=str(e)) from e
    except Exception as e: # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced (not
swallowed)
        raise HTTPException(status_code=502, detail=f"Could not resolve the source video: {e}")
from e
    if not os.path.isfile(local):
        raise HTTPException(status_code=404, detail="Source video file not found.")

-   # 3) Warm a proxy in the background for next time (local source only).
-   if is_local:
-       threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True).start()
+   # 3) Warm a proxy in the background for next time. Remote sources now warm too ? the
+   #   source-cache copy is a plain local file (closes the 'remote-source proxying' follow-
up).
+   threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True).start()

    media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
    return FileResponse(local, media_type=media)
diff --git a/tests/test_api_annotations.py b/tests/test_api_annotations.py
index c212810..cfe96d8 100644
--- a/tests/test_api_annotations.py
+++ b/tests/test_api_annotations.py
@@ -2,6 +2,9 @@
    GET /api/source/{video_id} (serve a video's source for annotation) and
    POST /api/annotations (persist finished rally labels as the byte-verbatim annotator CSV)."""

+import threading
+import time
+
+import pytest

    pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
@@ -10,6 +13,7 @@
    import backend.main as m
    from backend.config.models import AppConfig
    from backend.infrastructure.database import Database
+from backend.storage.ref import StorageStat

    class _InlineExecutor:
@@ -64,6 +68,151 @@ def test_source_serves_the_proxy_when_one_is_ready(tmp_path, monkeypatch):

```

```

assert r.content == b"PROXY-bytes" # proxy wins over source

+# ---- GET /api/source/{video_id} ? REMOTE-backed sources (R1-03: download ONCE, reuse) ----
+_REMOTE = b"remote-bytes"
+
+
+class _StubBackend:
+    """A stat-capable storage stub so the free-space guard can size the object without any
+    SDK."""
+
+    def __init__(self, size=None):
+        self._size = len(_REMOTE) if size is None else size
+
+    def stat(self, ref):
+        return StorageStat(size=self._size)
+
+
+def _counting_fetch(counter, payload=_REMOTE, delay=0.0):
+    """A storage.fetch double that counts downloads and writes ``payload`` to ``dst``."""
+
+    def _fetch(uri, dst=None, *, config=None, overwrite=False):
+        counter["n"] += 1
+        counter["uri"] = uri
+        if delay:
+            time.sleep(delay)
+        with open(dst, "wb") as fh:
+            fh.write(payload)
+        return str(dst)
+
+    return _fetch
+
+
+def test_source_remote_is_fetched_once_and_reused(tmp_path, monkeypatch):
+    """R1-03: FileResponse serves Range, so a scrubbing browser hits this URL many times ? the
+    remote object must be downloaded ONCE into the source cache and served from there after."""
+    client, db = _client(tmp_path, monkeypatch)
+    db.add_video("beefcafe", "gs://bucket/videos/match.mp4", "processed", [])
+    counter = {"n": 0}
+    monkeypatch.setattr(m.storage, "fetch", _counting_fetch(counter))
+    monkeypatch.setattr(m.storage, "get_storage", lambda *a, **k: _StubBackend())
+
+    r1 = client.get("/api/source/beefcafe")
+    r2 = client.get("/api/source/beefcafe", headers={"Range": "bytes=0-3"})
+    assert r1.status_code == 200 and r1.content == _REMOTE
+    assert r2.status_code == 206 and r2.content == _REMOTE[:4] # Range served from the cache
+    assert counter["n"] == 1 # ONE download total, both requests answered
+    assert counter["uri"] == "gs://bucket/videos/match.mp4"
+    cached = tmp_path / "source_cache" / "beefcafe" / "match.mp4"
+    assert cached.read_bytes() == _REMOTE # cached under output_base, keyed by video_id
+
+
+def test_source_local_path_stays_identity_passthrough(tmp_path, monkeypatch):
+    """A local-path video is served straight off its recorded path ? no fetch, no cache dir."""
+    client, db = _client(tmp_path, monkeypatch)
+    vid = tmp_path / "match.mp4"
+    vid.write_bytes(b"local-bytes")
+    db.add_video("localvid", str(vid), "processed", [])
+
+    def _boom(*a, **k):
+        raise AssertionError("fetch must not be called for a local source")
+
+    monkeypatch.setattr(m.storage, "fetch", _boom)
+
+    r = client.get("/api/source/localvid")

```

```

+     assert r.status_code == 200 and r.content == b"local-bytes"
+     assert not (tmp_path / "source_cache").exists() # nothing was copied or cached
+
+
+def test_source_remote_stale_partial_is_never_served(tmp_path, monkeypatch):
+    """A crashed/in-flight download leaves only ``*.partial`` names; the endpoint must never
+    serve one ? it re-fetches and atomically publishes the complete copy."""
+    client, db = _client(tmp_path, monkeypatch)
+    db.add_video("cafe0123", "gs://bucket/videos/match.mp4", "processed", [])
+    vid_dir = tmp_path / "source_cache" / "cafe0123"
+    vid_dir.mkdir(parents=True)
+    (vid_dir / ".match.mp4.deadbeef.partial").write_bytes(b"TRUNCATED")
+    counter = {"n": 0}
+    monkeypatch.setattr(m.storage, "fetch", _counting_fetch(counter))
+    monkeypatch.setattr(m.storage, "get_storage", lambda *a, **k: _StubBackend())
+
+    r = client.get("/api/source/cafe0123")
+    assert r.status_code == 200
+    assert r.content == _REMOTE # the complete fetch ? never the truncated partial
+    assert counter["n"] == 1
+    assert (vid_dir / "match.mp4").read_bytes() == _REMOTE
+
+
+def test_source_remote_fetch_failure_is_502_and_leaves_no_partial(tmp_path, monkeypatch):
+    """A failed download surfaces as 502 and cleans its partial, so a retry starts clean."""
+    client, db = _client(tmp_path, monkeypatch)
+    db.add_video("fail0123", "gs://bucket/videos/match.mp4", "processed", [])
+    monkeypatch.setattr(m.storage, "get_storage", lambda *a, **k: _StubBackend())
+
+    def _broken(*a, **k):
+        raise RuntimeError("bucket unreachable")
+
+    monkeypatch.setattr(m.storage, "fetch", _broken)
+    assert client.get("/api/source/fail0123").status_code == 502
+    vid_dir = tmp_path / "source_cache" / "fail0123"
+    assert not any(vid_dir.iterdir()) # no partial (or anything else) left behind
+
+    counter = {"n": 0}
+    monkeypatch.setattr(m.storage, "fetch", _counting_fetch(counter))
+    r = client.get("/api/source/fail0123") # the retry succeeds and caches normally
+    assert r.status_code == 200 and r.content == _REMOTE and counter["n"] == 1
+
+
+def test_source_remote_download_is_free_space_guarded(tmp_path, monkeypatch):
+    """D11: an object the disk can't hold is refused with 507 BEFORE any bytes are pulled."""
+    client, db = _client(tmp_path, monkeypatch)
+    db.add_video("big00001", "gs://bucket/videos/huge.mp4", "processed", [])
+    monkeypatch.setattr(
+        m.storage, "get_storage", lambda *a, **k: _StubBackend(size=10**18)
+    )
+
+    def _boom(*a, **k):
+        raise AssertionError("must not fetch when the disk cannot hold the object")
+
+    monkeypatch.setattr(m.storage, "fetch", _boom)
+    assert client.get("/api/source/big00001").status_code == 507
+
+
+def test_source_concurrent_first_requests_download_once(tmp_path, monkeypatch):
+    """Two racing first requests serialize on the per-video lock: ONE download, both served."""
+    cfg = AppConfig()
+    cfg.output.output_dir = str(tmp_path)
+    counter = {"n": 0}
+    monkeypatch.setattr(m.storage, "fetch", _counting_fetch(counter, delay=0.2))
+    monkeypatch.setattr(m.storage, "get_storage", lambda *a, **k: _StubBackend())

```

```

+
+ results = []
+
+ def _hit():
+     results.append(
+         m._acquire_source_for_serving("race0001", "gs://bucket/videos/match.mp4", cfg)
+     )
+
+ threads = [threading.Thread(target=_hit) for _ in range(2)]
+ for t in threads:
+     t.start()
+ for t in threads:
+     t.join()
+ assert counter["n"] == 1 # the loser waited, re-checked, and reused the winner's copy
+ assert len(results) == 2 and results[0] == results[1]
+ with open(results[0], "rb") as fh:
+     assert fh.read() == _REMOTE
+
+
+ # ---- POST /api/annotations ----
+ def test_annotations_persist_verbatim_csv(tmp_path, monkeypatch):
+     client, _ = _client(tmp_path, monkeypatch)

```

#### 4. assistant (2026-07-02T08:18:01.057Z)

Now let me verify the master code the analyst cited.

#### 5. user (2026-07-02T08:18:02.672Z)

```

595
596     # In-flight annotation-proxy generations ? dedup so a browser's many Range requests to
/api/source
597     # don't each kick off a transcode.
598     _proxy_inflight: "set[str]" = set()
599     _proxy_lock = threading.Lock()
600     _source_cache_locks: Dict[str, threading.Lock] = {}
601     _source_cache_locks_guard = threading.Lock()
602
603
604     def _proxies_dir(cfg) -> str:
605         return os.path.join(output_base(cfg), "proxies")
606
607
608     def _source_cache_dir(cfg) -> str:
609         return os.path.join(output_base(cfg), "source_cache")
610
611
612     def _report_config_for_input(config: Any, raw_input: str) -> Any:
613         """Return report config that keeps remote-input reports out of disposable
scratch."""
614         storage_config = (
615             config.get("storage") if isinstance(config, dict) else getattr(config,
"storage", None)
616         )
617         try:
618             if storage.input_is_local(raw_input, storage_config):
619                 return config
620         except ValueError:
621             return config
622         try:
623             cfg = (
624                 config.model_dump(mode="json")
625                 if hasattr(config, "model_dump")
626                 else dict(config or {})
627             )

```

```

628     except Exception:
629         cfg = {}
630     report = dict(cfg.get("report") or {})
631     if report.get("output_dir"):
632         return config
633     report["output_dir"] = os.path.join(output_base(config), "reports")
634     cfg["report"] = report
635     return cfg
636
637
638 def _source_cache_lock(video_id: str) -> threading.Lock:
639     with _source_cache_locks_guard:
640         return _source_cache_locks.setdefault(video_id, threading.Lock())
641
642
643 def _source_cache_path(video_id: str, input_ref, cfg) -> str:
644     ext = os.path.splitext(input_ref.name or "")[1].lower()
645     if not ext or "/" in ext or "\\" in ext or len(ext) > 16:
646         ext = ".mp4"
647     cache_dir = _source_cache_dir(cfg)
648     name = safe_output_filename(f"{video_id}{ext}")
649     return resolve_under_root(os.path.join(cache_dir, name), cache_dir)
650
651
652 def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
653     """Fetch a remote source once into an output-scoped source/proxy cache."""
654     cache_path = _source_cache_path(video_id, input_ref, cfg)
655     if os.path.isfile(cache_path):
656         return cache_path
657     lock = _source_cache_lock(video_id)
658     with lock:
659         if os.path.isfile(cache_path):
660             return cache_path
661         cache_dir = os.path.dirname(cache_path)
662         os.makedirs(cache_dir, exist_ok=True)
663         need = _remote_input_size_bytes(input_ref, getattr(cfg, "storage", None))
664         require_free_bytes(cache_dir, need, stage="source-cache")
665         tmp = os.path.join(
666             cache_dir, f".{os.path.basename(cache_path)}.{uuid.uuid4().hex}.tmp"
667         )
668         try:
669             storage.fetch(input_ref.uri, tmp, config=_storage_settings(cfg))
670             os.replace(tmp, cache_path)
671         finally:
672             try:
673                 if os.path.exists(tmp):
674                     os.remove(tmp)
675             except OSError:
676                 pass
677         return cache_path
678
679
680 def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
681     """Background, best-effort: generate the annotation proxy ONCE so subsequent
682     /api/source loads
683     serve the lighter, seekable copy. Deduped; never blocks the request; the source
684     keeps serving if
685     it fails."""
686     with _proxy_lock:
687         if video_id in _proxy_inflight:
688             return
689         _proxy_inflight.add(video_id)
690     try:
691         from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
692         if not needs_proxy(src_local):

```



```

691         return # already light enough ? no proxy worth making
692         make_annotation_proxy(src_local, os.path.join(_proxies_dir(cfg),
f"{video_id}.mp4"))
693     except Exception as e: # noqa: BLE001 - warming is best-effort
694         logger.warning("proxy: warm failed for %s: %s", video_id, e)
695     finally:
696         with _proxy_lock:
697             _proxy_inflight.discard(video_id)
698
699
700     @app.get("/api/source/{video_id}")
701     def get_source(video_id: str, request: Request):
702         """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub.
Serves a
703         frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels
still keyed
704         to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy
in the
705         background for next time ? non-blocking, so the first load is never delayed. Remote
sources are
706         fetched once into an output-scoped source cache, then served and proxied from that
stable copy. """
707         rec = request.app.state.db.get_video(video_id)
708         if not rec or not rec.get("filepath"):
709             raise HTTPException(status_code=404, detail="Video not found.")
710         cfg = request.app.state.config
711
712         # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
713         try:
714             proxy_name = safe_output_filename(f"{video_id}.mp4")
715             proxy_path = resolve_under_root(os.path.join(_proxies_dir(cfg), proxy_name),
_proxies_dir(cfg))
716         except ValueError:
717             proxy_path = ""
718         if proxy_path and os.path.isfile(proxy_path):
719             return FileResponse(proxy_path, media_type="video/mp4")
720
721         # 2) Otherwise resolve + serve the source.
722         try:
723             input_ref = storage.resolve_input_ref(
724                 rec["filepath"], getattr(cfg, "storage", None)
725             )
726         except ValueError as e: # unsupported scheme etc. ? a clean client error, never a
500
727             raise HTTPException(status_code=400, detail=str(e)) from e
728         except Exception as e: # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced
(not swallowed)
729             raise HTTPException(status_code=502, detail=f"Could not resolve the source
video: {e}") from e
730
731         if input_ref.backend == "local":
732             local = input_ref.key
733         else:
734             try:
735                 local = _cached_remote_source(video_id, input_ref, cfg)
736             except HTTPException:
737                 raise
738             except Exception as e: # noqa: BLE001 - a fetch miss is surfaced (not
swallowed)
739                 raise HTTPException(
740                     status_code=502, detail=f"Could not resolve the source video: {e}"
741                 ) from e
742         if not os.path.isfile(local):
743             raise HTTPException(status_code=404, detail="Source video file not found.")
744

```

```

745         # 3) Warm a proxy in the background for next time.
746         threading.Thread(
747             target=_warm_annotation_proxy, args=(video_id, local, cfg), daemon=True
748         ).start()
749
750         media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
751         return FileResponse(local, media_type=media)
752
753
754     @app.post("/api/annotations")

```

## 6. assistant (2026-07-02T08:18:05.857Z)

Master implements the full R1-03 intent. Let me verify the supporting helpers and the master test to confirm complete coverage of what the PR tests assert.

## 7. user (2026-07-02T08:18:08.727Z)

```

def require_free_bytes(
    path: str, need: Optional[int], *, stage: str = "write"
) -> None:
    """Raise HTTP 507 if ``path``'s filesystem can't hold ``need`` bytes; otherwise return.

    ``need`` of ``None`` or ``<= 0`` is a no-op (nothing to reserve). Measuring is best-effort:
an
    undeterminable free-space read logs a warning and proceeds (we never deny on inability to
    measure). A confirmed shortfall raises so the caller fails fast with an actionable 507."""
    if not need or need <= 0:
        return
    free = free_bytes(path)
    if free is None:
        ---paths---
def resolve_under_root(path: str, root: str) -> str:
    """Return the real path, asserting it is contained under `root` (raises ValueError if
not)."""
    rp = os.path.realpath(path)
    rr = os.path.realpath(root)
    try:
        contained = os.path.commonpath([rp, rr]) == rr
    except ValueError: # different drives on Windows
        contained = False
    if not contained:
        raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
    return rp

def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
    """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.

    ---remote_input_size---
        req.video_path, local_video, getattr(config, "storage", None)
    )

def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
    """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.

    Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat that needs
a
    missing cloud SDK, the network, or a credential returns ``None`` so the free-space guard
degrades
    to a no-op rather than blocking a job it cannot measure."""
    try:
        sc = (
            storage_config.model_dump()

```

```

        if hasattr(storage_config, "model_dump")
        else (storage_config or {}))
    )
    backend = storage.get_storage(input_ref.backend, config=sc)
    return int(backend.stat(input_ref).size)
except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must not block
    logger.debug(
        "free-space: could not stat remote source %s: %s", input_ref.uri, e
    )
return None

```

```
@app.post("/api/process", status_code=202)
```

## 8. assistant (2026-07-02T08:18:12.915Z)

All supporting helpers confirmed on master. Now let me verify the master test to confirm behavioral coverage matching the PR's test intent.

## 9. user (2026-07-02T08:18:13.977Z)

```

60         (proxies / "vidp.mp4").write_bytes(b"PROXY-bytes") # a ready proxy (output_dir ==
tmp_path)
61
62         r = client.get("/api/source/vidp")
63         assert r.status_code == 200
64         assert r.content == b"PROXY-bytes" # proxy wins over source
65
66
67     def test_source_caches_remote_fetch_and_warms_proxy_from_cache(tmp_path, monkeypatch):
68         """P3: repeated source/Range requests for a remote video reuse one local source
cache."""
69         client, db = _client(tmp_path, monkeypatch)
70         db.add_video("vidremote", "gs://bucket/videos/match.mp4", "processed", [])
71         payload = b"REMOTE-video-bytes"
72         fetches = []
73         warmed = []
74
75         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
76             fetches.append((uri, dst))
77             with open(dst, "wb") as f:
78                 f.write(payload)
79             return dst
80
81         class _InlineThread:
82             def __init__(self, target, args=(), kwargs=None, daemon=None):
83                 self.target = target
84                 self.args = args
85                 self.kwargs = kwargs or {}
86
87             def start(self):
88                 self.target(*self.args, **self.kwargs)
89
90         monkeypatch.setattr(m.storage, "fetch", fake_fetch)
91         monkeypatch.setattr(m, "_remote_input_size_bytes", lambda *a, **k: len(payload))
92         monkeypatch.setattr(m.threading, "Thread", _InlineThread)
93         monkeypatch.setattr(
94             m,
95             "_warm_annotation_proxy",
96             lambda video_id, src_local, cfg: warmed.append((video_id, src_local)),
97         )
98
99         first = client.get("/api/source/vidremote", headers={"Range": "bytes=0-5"})
100        second = client.get("/api/source/vidremote", headers={"Range": "bytes=6-11"})
101

```

```

102     assert first.status_code in (200, 206)
103     assert second.status_code in (200, 206)
104     assert len(fetches) == 1
105     assert fetches[0][0] == "gs://bucket/videos/match.mp4"
106     cache_path = tmp_path / "source_cache" / "vidremote.mp4"
107     assert cache_path.read_bytes() == payload
108     assert warmed == [("vidremote", str(cache_path)), ("vidremote", str(cache_path))]
109
110
111     # ---- POST /api/annotations ----
112     def test_annotations_persist_verbatim_csv(tmp_path, monkeypatch):
113         client, _ = _client(tmp_path, monkeypatch)
114         csv = (
115             "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
116             "1,8.800,11.500,winner,badminton,9\n"
117             "2,24.389,46.589,unforced_error,badminton,21\n"
118         )
119         r = client.post("/api/annotations", json={"video_id": "abc123", "csv": csv})
120         assert r.status_code == 200
121         body = r.json()
122         assert body == {"status": "ok", "num_rows": 2, "csv_uri":
"labels/abc123.rallies.csv"}
123
124         written = tmp_path / "labels" / "abc123.rallies.csv"
125         assert written.read_bytes() == csv.encode("utf-8") # byte-verbatim (pipeline join
needs it exact)
126
127
128     def test_annotations_reject_unsafe_video_id(tmp_path, monkeypatch):
129         client, _ = _client(tmp_path, monkeypatch)
130         r = client.post("/api/annotations", json={"video_id": "../evil", "csv": "x"})
131         assert r.status_code == 400
132         assert not (tmp_path.parent / "evil.rallies.csv").exists() # nothing escaped the
labels dir
133

```

## 10. assistant (2026-07-02T08:18:26.741Z)

I have thoroughly verified the analyst's claims. Let me assess the one difference worth scrutinizing: the PR's cache path keying.

The PR caches at `source\_cache/<video\_id>/<name>` (per-video subdir, keyed by remote object name). Master caches at `source\_cache/<video\_id><ext>` (flat, keyed by video\_id + extension). These are functionally equivalent for the R1-03 intent (fetch-once, reuse across Range requests). Both:

- Local passthrough (identity, no fetch): master `input\_ref.key` at 731-732 vs PR `\_acquire\_source\_for\_serving` local branch
- Fetch-once with double-checked lock: master 655-660 vs PR
- Free-space 507 guard before fetch: master `require\_free\_bytes(cache\_dir, need, stage="source-cache")` at 664 vs PR
- Atomic publish via `os.replace` + partial cleanup: master 665-676 vs PR
- 502-on-failure with HTTPException re-raise: master 736-741 vs PR
- Unconditional proxy warming for remote too: master 745-748 vs PR

Master additionally uses `resolve\_under\_root` for path-escape safety (PR does not) and avoids trusting the remote object name (master derives filename from `video\_id`). Both PR test scenarios (stale-partial, concurrent-race, free-space-guard) map to behaviors master already implements ? master has a slightly narrower test (one test vs PR's six), but all the \*behaviors\* the PR tests assert are present and correct on master.

The PR diff's base (`storage.acquire\_local\_input`/`input\_is\_local`) no longer exists in master's `get\_source` ? confirming it cannot apply cleanly and is same-region supersession, not a divergent capability.

The analyst is correct. The only edge worth noting: the PR carries broader test coverage (6

tests including explicit stale-partial and concurrency tests), but these exercise behaviors master already implements correctly; no unique guard or capability is lost by closing.

**11. user (2026-07-02T08:18:34.588Z)**

Structured output provided successfully